CSA1711 - Artificial Intelligence

CO2 Assessment Tool - 3

Dry Run Evaluation - Answer Script

Question 1: Breadth-First Search (BFS)

Graph structure used: *A is the root, with children B and C. B has children D and E. C has children F and G.*

Task: Starting from node A, perform a Breadth-First Search (BFS) using a FIFO queue. The dry run below shows the queue contents and the node visited at each iteration.

BFS Dry Run Table

Iteration	Queue (after processing)	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	(empty)	G

Answers

1. BFS Traversal Order: A, B, C, D, E, F, G

2. Queue contents after each iteration: Iteration 1 - Queue: [B, C] (A dequeued and visited, its children B and C enqueued); Iteration 2 - Queue: [C, D, E] (B dequeued and visited, its children D and E enqueued); Iteration 3 - Queue: [D, E, F, G] (C dequeued and visited, its children F and G enqueued); Iteration 4 - Queue: [E, F, G] (D dequeued and visited, D has no children); Iteration 5 - Queue: [F, G] (E dequeued and visited, E has no children); Iteration 6 - Queue: [G] (F dequeued and visited, F has no children); Iteration 7 - Queue: [] (G dequeued and visited, G has no children - queue is now empty and BFS terminates).

Question 2: Depth-First Search (DFS)

Graph structure used: *A is the root, with children B and C. B has children D and E. C has children F and G.*

Task: Using the same graph, perform a Depth-First Search (DFS) starting from node A, using a LIFO stack. When a node's children are pushed, they are pushed so that the left-most child (in reading order) is popped and visited first.

DFS Dry Run Table

Iteration	Stack (after processing)	Visited Node
1	C, B	A
2	C, E, D	B
3	C, E	D
4	C	E
5	G, F	C
6	G	F
7	(empty)	G

Answers

DFS Traversal Order: A, B, D, E, C, F, G

Explanation of stack evolution: Iteration 1 - A is popped and visited; its children C and B are pushed (B on top so it is explored first), giving Stack: [C, B]; Iteration 2 - B is popped and visited; its children E and D are pushed (D on top), giving Stack: [C, E, D]; Iteration 3 - D is popped and visited; D has no children, giving Stack: [C, E]; Iteration 4 - E is popped and visited; E has no children, giving Stack: [C]; Iteration 5 - C is popped and visited; its children G and F are pushed (F on top), giving Stack: [G, F]; Iteration 6 - F is popped and visited; F has no children, giving Stack: [G]; Iteration 7 - G is popped and visited; G has no children - stack is now empty and DFS terminates.